

Lab 8

3. Multithreading in Java (II)

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Tuesday, March 10th, 2026 | | ER 3120



University of Windsor

● Outline

- • Executors
- Callable & Futures
- Fork-Join Framework



Part I

Multiple Threads in Java





Executors



Executors

- Executors provide a high-level interface for managing a pool of threads.
- Simplify the execution of tasks in asynchronous mode.
- Java provides several types of executor services, including single thread executor, fixed thread pool, and scheduled thread pool.



Executors

- **Simplified Task Execution Management:** Executors abstract away the complexities of manually creating, managing, and monitoring threads. You can submit tasks to be executed without dealing with the lifecycle of threads.
- **Resource Reuse:** By maintaining a pool of threads, executors reuse existing threads for multiple tasks. This reduces the overhead of thread creation and teardown, leading to more efficient execution, especially for short-lived asynchronous tasks.
- **Future and Callable Support:** Executors work seamlessly with the Future and Callable interfaces, enabling tasks to return results after completion and allowing the application to effectively handle asynchronous task results.



Executors: An Example



Example: Executors

```
1 import java.util.concurrent.ExecutorService;
2 import java.util.concurrent.Executors;
3
4 public class ExecutorsExample {
5     Run | Debug
6     public static void main(String[] args) {
7         // Creates a thread pool with a fixed number of threads
8         ExecutorService executor = Executors.newFixedThreadPool(3);
9
10        // Submit tasks to the executor
11        executor.submit(() -> {
12            System.out.println("Task 1 executed by " + Thread.currentThread().getName());
13        });
14
15        executor.submit(() -> {
16            System.out.println("Task 2 executed by " + Thread.currentThread().getName());
17        });
18
19        executor.submit(() -> {
20            System.out.println("Task 3 executed by " + Thread.currentThread().getName());
21        });
22
23        // Shut down the executor once the tasks are completed
24        executor.shutdown();
25    }
26 }
```

ExecutorService Creation:

- An ExecutorService named executor is created using **Executors.newFixedThreadPool(3)**.
- This means the thread pool will contain 3 threads to execute tasks.

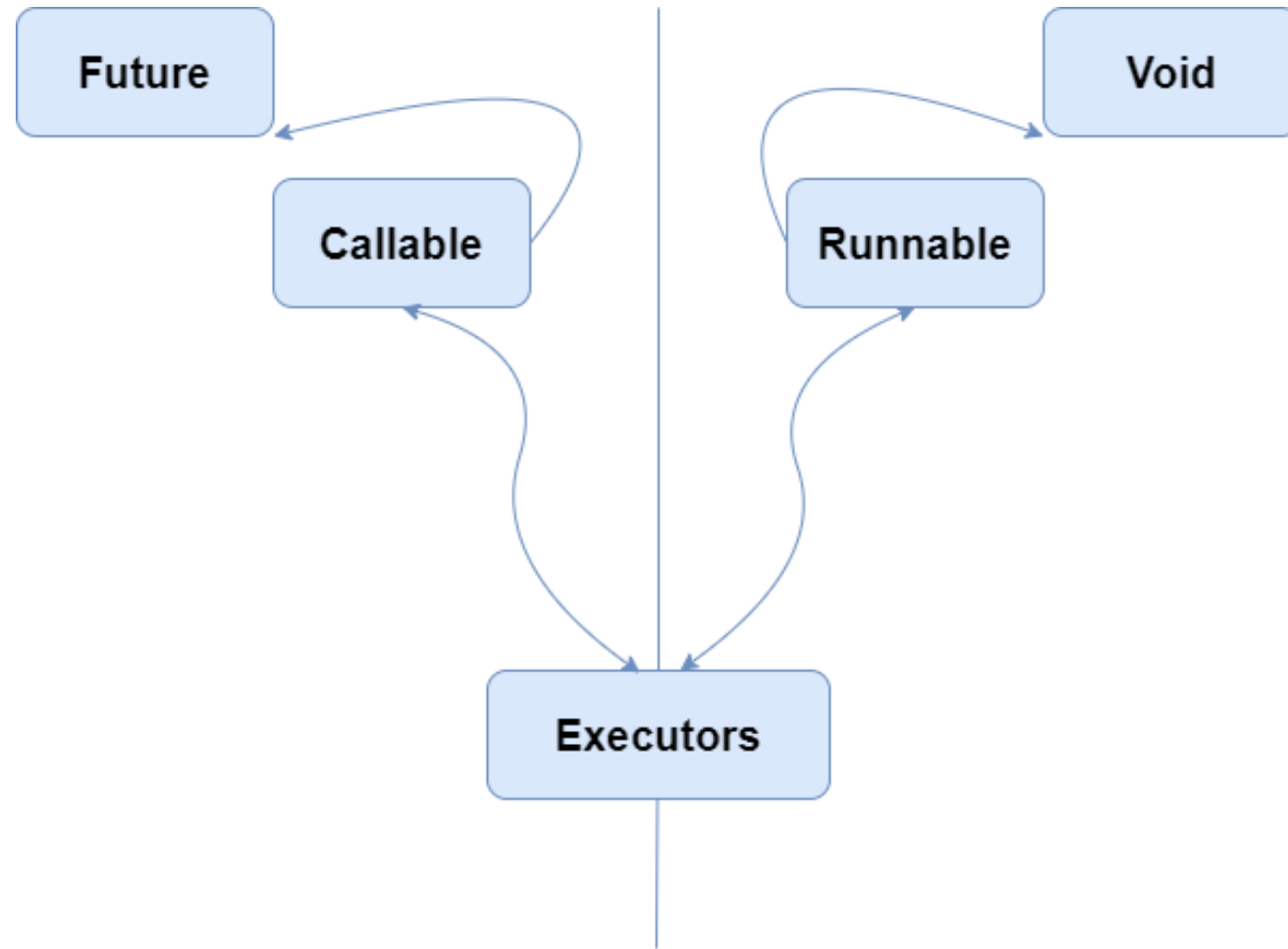
Task Submission:

- Three tasks are submitted to the executor using lambda expressions.
- Each task is an implementation of the Runnable interface, indicated by the **() -> {}** syntax.

ExecutorService Shutdown:

- After submitting the tasks, **executor.shutdown()** is called to initiate an orderly shutdown in which previously submitted tasks are executed, but no new tasks will be accepted.

Executors: Process Flow



Runnable: Review



Review: Runnable

- Type of class (Runnable is an Interface) that can be put into a thread, describing what the thread is supposed to do.
- Executes tasks without returning a result or throwing checked exception:
 - Runnable does not permit the method `run()` to throw any checked exceptions. Any exceptions thrown inside the `run()` method must be caught and handled within the method itself.



Example: Runnable

```
1  public class MyRunnableTask implements Runnable {
2      @Override
3      public void run() {
4          // Task code here
5      }
6
7      Run | Debug
8      public static void main(String[] args) {
9          Thread t = new Thread(new MyRunnableTask());
10         t.start();
11     }
12
13     // public static void main(String[] args) {
14     //     Thread t = new Thread(new Runnable() {
15     //         @Override
16     //         public void run() {
17     //             // Inline task code
18     //         }
19     //     });
20     //     t.start();
21     // }
```



Callable



University of Windsor

● Callable



- An interface similar to `Runnable`, but it allows the task to return a value and to throw an exception.
- The `Callable` interface is a generic type, which means you define the type of object that the `Callable` returns.



Example: Callable

```
1  import java.util.concurrent.Callable;
2
3  public class WordLengthCallable implements Callable<Integer> {
4      private final String word;
5
6      public WordLengthCallable(String word) {
7          this.word = word;
8      }
9
10     @Override
11     public Integer call() throws Exception {
12         return word.length();
13     }
14 }
```

Part II

Java Wrapper Classes





Wrapper Classes



Review: Java Wrapper Classes

- Java wrapper classes are used to convert primitive data types into objects.
- These classes "*wrap*" the primitive in an object, so they can be included in activities reserved for objects, like being added to Collections.
- Each primitive data type has a corresponding wrapper class:
 - Boolean for boolean
 - Byte for byte
 - Character for char
 - Double for double
 - Float for float
 - Integer for int
 - Long for long
 - Short for short



```
public class WordLengthCallable implements Callable<Integer>
```



The Future object



● The Future object

- To execute `Callable` tasks, you use an `ExecutorService`.
- When you submit a `Callable` to an executor, you receive a `Future` object.
- This `Future` represents the pending result of the `Callable` and can also be used to check if the `Callable` has finished executing and to retrieve the result.
- `Future` also provides other useful methods like:
 - `isDone()` to check if the task is complete,
 - `cancel()` to cancel the task, and
 - `isCancelled()` to check if the task was cancelled.

Example: Future

```
1  import java.util.concurrent.*;
2
3  public class CallableFutureExample {
4      public static void main(String[] args) {
5          ExecutorService executor = Executors.newFixedThreadPool(nThreads:1);
6
7          // Create a Callable instance
8          Callable<Integer> callableTask = new WordLengthCallable(word:"Hello, World!");
9
10         // Submit the Callable to the executor
11         Future<Integer> future = executor.submit(callableTask);
12
13         try {
14             // Retrieve the result of the Callable
15             Integer result = future.get(); // This line blocks until the result is available
16             System.out.println("Word length: " + result);
17         } catch (InterruptedException | ExecutionException e) {
18             e.printStackTrace();
19         }
20
21         executor.shutdown();
22     }
23 }
```

Part III

The Fork / Join Framework



The Fork-Join Framework



● The Fork-Join Framework

- • Designed to efficiently execute tasks that can be broken down into smaller subtasks, following a divide-and-conquer strategy.
- Useful for taking advantage of multi-core processors.
- Main concepts:
 - Work-Stealing Algorithm
 - ForkJoinPool
 - ForkJoinTask
 - Forking and Joining



● Work-Stealing Algorithm

- It uses a work-stealing algorithm where idle threads can "steal" work from those with pending tasks.
- This helps in efficiently balancing the workload across all available processor cores.

ForkJoinPool

- The heart of the framework is the `ForkJoinPool` class, a specialized implementation of the `ExecutorService`.
- When you submit tasks to the pool, it handles the details of task management, including the splitting of tasks and the assignment of those tasks to threads.

ForkJoinTask

- This is the base class for tasks used with the `ForkJoinPool`. There are two common subclasses:
- `RecursiveAction`: for tasks that do not return a result.
- `RecursiveTask<V>`: for tasks that return a result of type `V`.



● Forking and Joining

○ • **Forking:**

The process of splitting a large task into smaller subtasks which can be executed independently and in parallel.

• **Joining:**

After the subtasks are executed (or "forked"), the results of the subtasks are combined (or "joined") to produce the final result.

Example: Fork-Join

```
1  import java.util.concurrent.ForkJoinPool;
2  import java.util.concurrent.RecursiveTask;
3
4  public class SumTask extends RecursiveTask<Long> {
5      private static final int THRESHOLD = 1000;
6      private final int[] array;
7      private final int start;
8      private final int end;
9
10     public SumTask(int[] array, int start, int end) {
11         this.array = array;
12         this.start = start;
13         this.end = end;
14     }
15
16     @Override
17     protected Long compute() {
18         int length = end - start;
19         if (length <= THRESHOLD) { // Base case threshold
20             long sum = 0;
21             for (int i = start; i < end; i++) {
22                 sum += array[i];
23             }
24             return sum;
25         } else {
26             int middle = start + (length / 2);
27             SumTask leftTask = new SumTask(array, start, middle);
28             SumTask rightTask = new SumTask(array, middle, end);
29
30             leftTask.fork(); // Fork the task to compute left half asynchronously
31             long rightResult = rightTask.compute(); // Compute right half
32             long leftResult = leftTask.join(); // Wait for the left half and get its result
33
34             return leftResult + rightResult;
35         }
36     }
37 }
```

```
37
38
39     public static long getTotalSum(int[] array) {
40         ForkJoinPool pool = new ForkJoinPool();
41         return pool.invoke(new SumTask(array, start:0, array.length));
42
43         // Try with resources, introduced in Java 7, to automatically close the pool:
44
45         // try (ForkJoinPool pool = new ForkJoinPool()) {
46         //     return pool.invoke(new SumTask(array, 0, array.length));
47         // }
48
49         // Manual pool management:
50
51         // ForkJoinPool pool = new ForkJoinPool();
52         // try {
53         //     return pool.invoke(new SumTask(array, 0, array.length));
54         // } finally {
55         //     pool.shutdown();
56         // }
57
58     Run | Debug
59     public static void main(String[] args) {
60         int[] array = new int[1000000]; // Large array
61         for (int i = 0; i < array.length; i++) {
62             array[i] = i;
63         }
64
65         long startTime = System.currentTimeMillis();
66         long totalSum = getTotalSum(array);
67         long endTime = System.currentTimeMillis();
68
69         System.out.println("Total Sum: " + totalSum);
70         System.out.println("Time taken: " + (endTime - startTime) + " ms");
71     }
```

Part IV

Application to Lab 8



Application to Lab 8



Application to Lab 8

- Creating a new File object representing the directory specified by the first command-line argument:

```
File directory = new File(args[0]);
```

- Getting the size of the file (or directory) in bytes:

```
size = file.length();
```

- This line gets an array of File objects representing the files and directories contained within the directory represented by the file object:

```
File[] subFiles = file.listFiles();
```

Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor